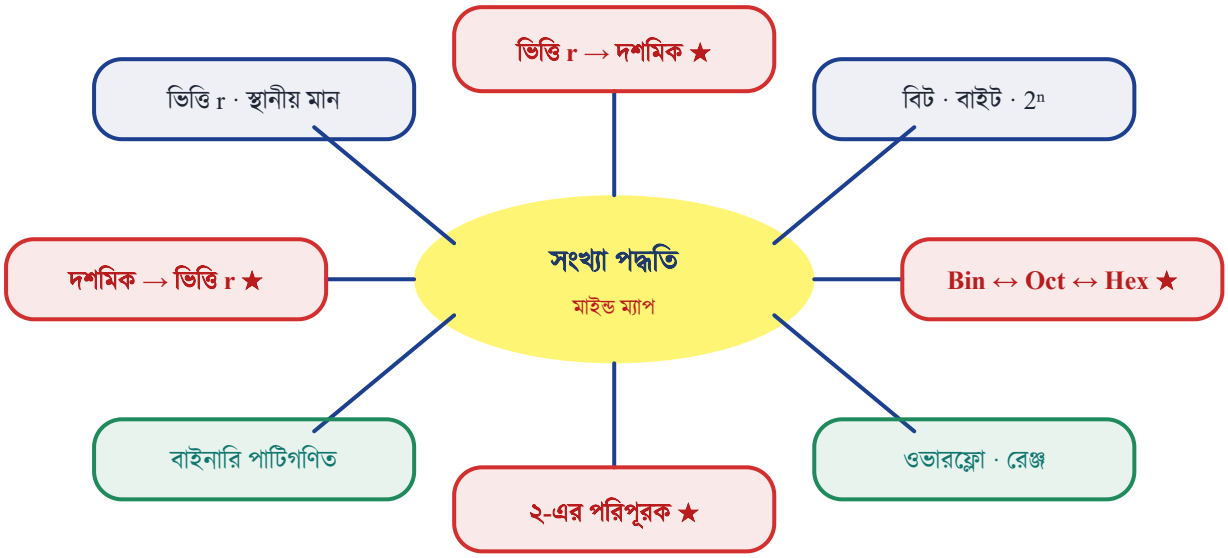
সংখ্যা পদ্ধতি (Number System)

বাইনারি · অক্টাল · হেক্স · রূপান্তর · পাটিগণিত · ২-এর পরিপূরক: ১১ পাতায় পরীক্ষার পূর্ণ প্রস্তুতি



↑ পুরো টপিক এক পাতায়: লাল = বেশি আসে, সবুজ = পদ্ধতি / প্রয়োগ

সিলেবাসের যে অংশ

"Introduction to Computer Systems: … Basic Information Technology, The Internet, **Number system: binary, octal, hexadecimal, binary arithmetic**"। এই নোটে আছে পজিশনাল সংখ্যা পদ্ধতি, প্রতিটি ভিত্তি-রূপান্তর (পূর্ণাংশ ও ভগ্নাংশ), ২, ৮ ও ১৬ ভিত্তিতে পাটিগণিত, পরিপূরক, রেঞ্জ ও ওভারফ্লো। কোড (BCD, Gray, Excess-3) ও চিহ্নযুক্ত সংখ্যার সার্কিট বিস্তারিত আছে DLD-এর সংখ্যা পদ্ধতি নোটে।

হাই-ইল্ড তালিকা

① 2^{16} পর্যন্ত ২-এর ঘাত
② ভগ্নাংশের রূপান্তর
③ ৩-বিট / ৪-বিট গ্রুপিং
④ হেক্স / অক্টাল যোগ-বিয়োগ
⑤ ২-এর পরিপূরক ও রেঞ্জ
⑥ ওভারফ্লোর নিয়ম

সংকেত: **হলুদ** = মূল শব্দ · **লাল বৃত্ত** = মুখস্থ রাখার তথ্য · **বেগুনি বক্স** = মনে রাখার কৌশল · **লাল বক্স** = পরীক্ষার ফাঁদ · **সবুজ** = সমাধানকৃত উদাহরণ

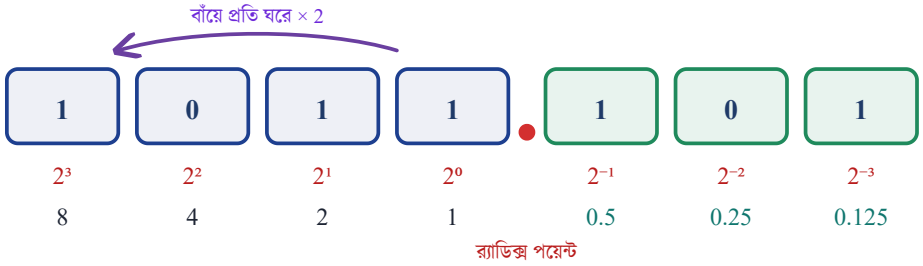
সূচিপত্র

✓ পজিশনাল পদ্ধতি, রূপান্তর-মানচিত্র … পৃ. ২
✓ বিট, 2^n, যেকোনো ভিত্তি → দশমিক … পৃ. ৩
✓ দশমিক → যেকোনো ভিত্তি ..... পৃ. ৪
✓ বাইনারি ↔ অক্টাল ↔ হেক্স ..... পৃ. ৫
✓ বাইনারি যোগ ও বিয়োগ ..... পৃ. ৬
✓ গুণ, ভাগ, অক্টাল/হেক্স হিসাব … পৃ. ৭
✓ পরিপূরক ও চিহ্নযুক্ত সংখ্যা … পৃ. ৮
✓ পরিপূরকে বিয়োগ, ওভারফ্লো … পৃ. ৯
✓ পরীক্ষার আগের রাতের পাতা … পৃ. ১০
✓ অনুশীলন MCQ + লিখিত ..... পৃ. ১১

১ পজিশনাল সংখ্যা পদ্ধতি

সংজ্ঞা: সংখ্যা পদ্ধতি হলো নির্দিষ্ট কিছু অঙ্ক (digit) ও একটি **ভিত্তি বা বেস (base/radix) r** দিয়ে সংখ্যা প্রকাশের নিয়ম; r = ব্যবহৃত ভিন্ন অঙ্কের মোট সংখ্যা, **0 সহ**। অঙ্কগুলো **0 থেকে r − 1**। **পজিশনাল (positional)** পদ্ধতিতে প্রতিটি অঙ্কের মান = অঙ্ক × $r^{\text{অবস্থান}}$।

মান: $N = d_{n-1}r^{n-1} + \ldots + d_1 r + d_0 + d_{-1}r^{-1} + d_{-2}r^{-2} + \ldots$ (d = অঙ্ক, r = ভিত্তি)



$(1011.101)_2 = 8 + 2 + 1 + 0.5 + 0.125 = 11.625$: প্রতিটি ঘরের মান তার ডান পাশের ঘরের r গুণ

পদ্ধতি	ভিত্তি	অঙ্ক	বড় অঙ্ক	উদাহরণ	অন্য লিখনরীতি
বাইনারি	2	0, 1	1	$(1011)_2$	1011B · 0b1011
অক্টাল	8	0 – 7	7	$(157)_8$	157Q · 0157 (C) · 0o157
দশমিক	10	0 – 9	9	$(245)_{10}$	245D
হেক্সাডেসিমাল	16	0 – 9, A – F	F = 15	$(2F)_{16}$	2FH · 0x2F

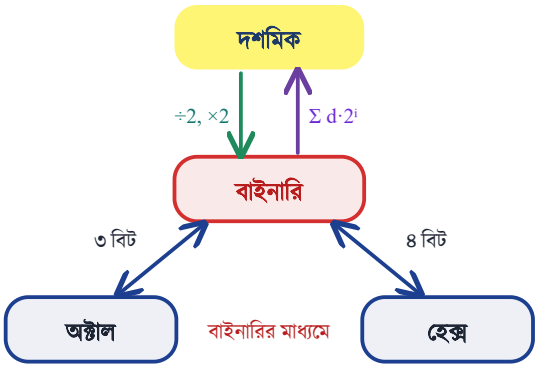
হেক্সের অক্ষর: **A = 10, B = 11, C = 12, D = 13, E = 14, F = 15**

কেন বাইনারি?: ইলেকট্রনিক যন্ত্রের **দুটি স্থিতিশীল অবস্থা** থাকে (অন/অফ, উচ্চ/নিম্ন ভোল্টেজ, চুম্বকিত বা নয়)। দুটি স্তর তৈরি করা সহজ, গুলিয়ে যাওয়ার ভয় কম, আর সরাসরি বুলিয়ান লজিকের সাথে মেলে।

পরীক্ষার ফাঁদ: যেকোনো ভিত্তিতে $(10)_r = r$: $10_2 = 2$, $10_8 = 8$, $10_{16} = 16$। রোমান সংখ্যা **নন-পজিশনাল**, এতে শূন্য নেই।

মনে রাখার কৌশল: "**ভিত্তি = কয়টি অঙ্ক; সবচেয়ে বড় অঙ্ক = ভিত্তি − 1**।" তাই অক্টালে ৮ বা ৯ এবং বাইনারিতে ২ কখনো আসে না: $(189)_8$ অবৈধ।

ডানের মানচিত্রটিই পুরো লেসন: পৃ. ৩ – ৫ প্রতিটি তীর পূর্ণাংশ ও ভগ্নাংশসহ দেখায়।



রূপান্তরের মানচিত্র

২ বিট, বাইট ও ২-এর ঘাত

- **বিট** = একটি বাইনারি অঙ্ক (0/1)
- **নিবল** = 4 বিট = **একটি হেক্স অঙ্ক**
- **বাইট** = 8 বিট = দুটি হেক্স অঙ্ক
- **ওয়ার্ড** = CPU-র স্বাভাবিক আকার (16/32/64 বিট)
- n বিট → **2^n** টি বিন্যাস, আনসাইনড $0 \dots 2^n - 1$
- n টি অ্যাড্রেস লাইন → 2^n টি লোকেশন: 16 লাইন × 8-বিট = 64 KB

n	2^n	n	2^n	n	2^n
0	1	6	64	12	4096
1	2	7	128	13	8192
2	4	8	**256**	14	16384
3	8	9	512	15	32768
4	16	10	**1024**	16	**65536**
5	32	11	2048	20	1048576

Dec	Bin	Oct	Hex	Dec	Bin	Oct	Hex
0	0000	0	0	8	1000	10	8
1	0001	1	1	9	1001	11	9
2	0010	2	2	10	1010	12	A
3	0011	3	3	11	1011	13	B
4	0100	4	4	12	1100	14	C
5	0101	5	5	13	1101	15	D
6	0110	6	6	14	1110	16	E
7	0111	7	7	15	1111	17	F

2^{10} = 1 K, 2^{20} = 1 M, 2^{30} = 1 G, 2^{40} = 1 T। ৮-এর ঘাত: 1, 8, 64, 512, 4096। ১৬-এর ঘাত: 1, 16, 256, 4096, 65536।

পরীক্ষার ফাঁদ: মেমরিতে 1 KB = **1024** বাইট (IEC: KiB)। SI / নেটওয়ার্কিং-এ 1 kB = 1000 বাইট, 1 kbps = 1000 bit/s।

৩ যেকোনো ভিত্তি → দশমিক

পদ্ধতি: প্রতিটি অঙ্কের নিচে তার **স্থানীয় মান (place value)** লেখো (বিন্দুর বাঁয়ে $r^0, r^1, \dots$; ডানে $r^{-1}, r^{-2}, \dots$), তারপর গুণ করে যোগ করো।

সমাধানকৃত উদাহরণ ✍️:

১. $(1011.101)_2 = 8 + 0 + 2 + 1 + 0.5 + 0 + 0.125 = \mathbf{11.625}$

২. $(157.4)_8 = 1\times64 + 5\times8 + 7\times1 + 4\times0.125 = 64 + 40 + 7 + 0.5 = \mathbf{111.5}$

৩. $(3A7)_{16} = 3\times256 + 10\times16 + 7 = 768 + 160 + 7 = \mathbf{935}$

৪. $(2F.8)_{16} = 2\times16 + 15 + 8/16 = \mathbf{47.5}$ · $(212)_3 = 2\times9 + 1\times3 + 2 = \mathbf{23}$

কৌশল: হর্নার: "**দ্বিগুণ করো, যোগ করো**", বাঁ থেকে ডানে। $(11001)_2$: 1 → 3 → 6 → 12 → **25**। ভিত্তি r হলে r দিয়ে গুণ করে পরের অঙ্ক যোগ করো।

দ্রুত যাচাই: শেষ অঙ্ক **0 হলে বাইনারি সংখ্যাটি জোড়**; সবচেয়ে বাঁয়ের 1 আকার বলে দেয়: $1xxxx_2$ থাকে 16 … 31-এর মধ্যে।

৪ দশমিক → যেকোনো ভিত্তি

পূর্ণাংশ: r দিয়ে বারবার **ভাগ** করো; প্রতিবার ভাগশেষ লেখো; ভাগফল 0 হলে থামো। ভাগশেষ পড়ো **নিচ → উপর** (প্রথম ভাগশেষ = LSD)।

ভগ্নাংশ: r দিয়ে বারবার **গুণ** করো; প্রতিবার পূর্ণাংশটি নাও; ভগ্নাংশ 0 হলে (বা যথেষ্ট অঙ্ক পেলে) থামো। পড়ো **উপর → নিচ** (প্রথমটি = MSD)।

2	245	ভাগশেষ
2	122	1
2	61	0
2	30	1
2	15	0
2	7	1
2	3	1
2	1	1
	0	1

উপরে পড়ো

$245 = (11110101)_2$

পূর্ণাংশের উদাহরণ ✍️:

১. $245 \div 8 \rightarrow$ ভাগশেষ **5, 6, 3** $\rightarrow$ $\mathbf{(365)_8}$

২. $245 \div 16 \rightarrow$ ভাগশেষ **5, 15 = F** $\rightarrow$ $\mathbf{(F5)_{16}}$

৩. $1000 \div 16 \rightarrow$ ভাগশেষ **8, 14 = E, 3** $\rightarrow$ $\mathbf{(3E8)_{16}}$

৪. $25 \rightarrow (11001)_2$ · যাচাই: $16 + 8 + 1 = 25$

শর্টকাট: ছোট সংখ্যায় **সবচেয়ে বড় ২-এর ঘাত বিয়োগ করো**: $100 = 64 + 32 + 4 \rightarrow (1100100)_2$।

ভগ্নাংশের উদাহরণ ✍️:

0.625 × 2	অঙ্ক	0.375 × 8	অঙ্ক	0.375 × 16	অঙ্ক
1.25	1	3.0	3	6.0	6
0.25 × 2 = 0.5	0	থামো (ভগ্নাংশ 0)		থামো	
0.5 × 2 = 1.0	1				

$0.625 = (0.101)_2$ · তাই $\mathbf{25.375 = (11001.011)_2 = (31.3)_8 = (19.6)_{16}}$

পরীক্ষার ফাঁদ: $(0.1)_{10} = (0.\mathbf{0011}\ 0011\ 0011\ \ldots)_2$ **কখনো শেষ হয় না**। কোনো ভগ্নাংশ বাইনারিতে সসীম হয় কেবল তখনই, যখন এর হর ২-এর ঘাত (0.5, 0.25, 0.375 …)। এ কারণেই ফ্লোটিং পয়েন্টে `0.1 + 0.2 != 0.3`।

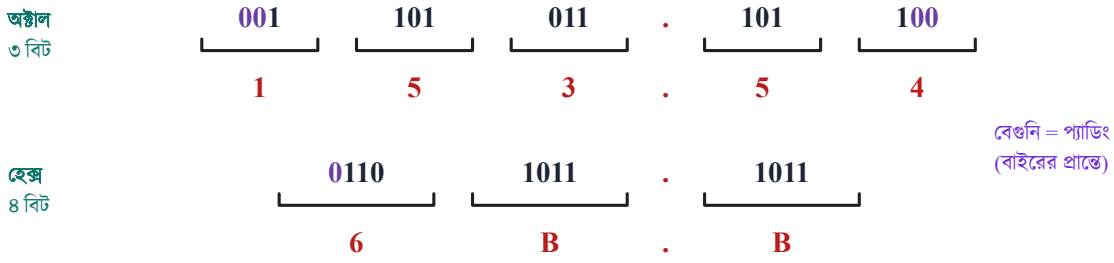
মনে রাখার কৌশল: "**ভাগ → উপরে পড়ো, গুণ → নিচে পড়ো**"। পূর্ণাংশ বিন্দু থেকে বাঁয়ে বাড়ে, ভগ্নাংশ বিন্দু থেকে ডানে।

ভিত্তি r → ভিত্তি s ✍️: **দশমিকের মাধ্যমে** যাও (দুটোই ২-এর ঘাত হলে বাইনারির মাধ্যমে): $(212)_3 \rightarrow 23 \rightarrow \mathbf{(10111)_2}$; $(0.6875)_{10} \times 8 \rightarrow 5.5, 4.0 \rightarrow \mathbf{(0.54)_8}$।

নির্ভুলতা: k টি ভগ্নাংশ-বিট 2^{-k} মাপের ধাপ আলাদা করতে পারে। 0.1 কে 8 বিটে কাটলে $= (0.00011001)_2 = 0.09765625$, তাই সংরক্ষিত মান **রাউন্ড** হয়।

৫ বাইনারি ↔ অক্টাল ↔ হেক্সাডেসিমাল

নিয়ম: $8 = 2^3$ এবং $16 = 2^4$, তাই **একটি অক্টাল অঙ্ক = ৩ বিট** এবং **একটি হেক্স অঙ্ক = ৪ বিট**। **র‍্যাডিক্স পয়েন্ট থেকে বাইরের দিকে** গ্রুপ করো: পূর্ণাংশের বিট বাঁয়ে, ভগ্নাংশের বিট ডানে; দরকার হলে 0 বসাও শুধু **বাইরের** প্রান্তে।



$$(1101011.1011)_2 = (153.54)_8 = (6B.B)_{16} = 107.6875$$

সমাধানকৃত উদাহরণ ✍️:

১. $(10110001)_2$: 10|110|001 = **$(261)_8$**; 1011|0001 = **$(B1)_{16}$**

২. $(157)_8$ → 001 101 111 → 0110 1111 = **$(6F)_{16}$** (= 111)

৩. $(3FA)_{16}$ → 0011 1111 1010 → 001 111 111 010 = **$(1772)_8$** (= 1018)

৪. $(A5)_{16}$ = 1010 0101 = $(245)_8$ = 165

পরীক্ষার ফাঁদ: ভগ্নাংশের বিটে প্যাডিং **ডানে**: .1011 → .101 100 = $.54_8$, .001 011 = $.13_8$ নয়। অক্টাল ↔ হেক্সে সরাসরি গ্রুপিং নেই: **বাইনারির মাধ্যমে** যাও।

মনে রাখার কৌশল: **"অক্টাল-৩, হেক্স-৪"**: ২-এর ঘাতই গ্রুপের আকার। ভিত্তি $4 = 2^2$ → ২ বিটের গ্রুপ; ভিত্তি $32 = 2^5$ → ৫ বিটের গ্রুপ।

অঙ্কের সংখ্যা: n-বিট সংখ্যায় লাগে **[n/3] টি অক্টাল** ও **[n/4] টি হেক্স** অঙ্ক: 32 বিট = ৮টি হেক্স অঙ্ক, ১১টি অক্টাল অঙ্ক।

৬ কোথায় কোন ভিত্তি

ভিত্তি	বাস্তব ব্যবহার	উদাহরণ
বাইনারি	মেশিন কোড, বিট মাস্ক, ফ্ল্যাগ	x & 0b1111 নিচের নিবল রাখে
অক্টাল	ইউনিক্স ফাইল পারমিশন (৩ বিট rwx)	chmod **755** = rwx r-x r-x
দশমিক	IPv4 ডটেড ডেসিমাল (৪টি অক্টেট)	192.168.1.1, প্রতিটি অংশ 0 – 255
হেক্সাডেসিমাল	মেমরি অ্যাড্রেস, MAC, IPv6, রং, ইউনিকোড	0x7FFF · #FF8800 · U+0985 (অ)

৭ বাইনারি যোগ

A + B (+ ক্যারি)	যোগফল	ক্যারি
0 + 0	0	0
0 + 1 = 1 + 0	1	0
$1 + 1 = (10)_2$	**0**	**1**
$1 + 1 + 1 = (11)_2$	**1**	**1**

	1	1	1	1	ক্যারি
		1	**0**	**1**	**1** = 11
+		**1**	**1**	**0**	**1** = 13
	1	**1**	**0**	**0**	**0** = 24 ✓

1011 + 1101 = 11000

আরও উদাহরণ ✍️:

১. $(101)_2 + (11)_2 = \mathbf{(1000)_2}$ (5 + 3 = 8)

২. $(101.11)_2 + (11.01)_2 = \mathbf{(1001.00)_2}$ (5.75 + 3.25 = 9): আগে বিন্দু বরাবর সাজাও

৩. তিন সারি: 111 + 101 + 11 = $\mathbf{(1111)_2}$ (7 + 5 + 3 = 15): কোনো কলামের যোগফল 3 = $(11)_2$ হলে 1 লেখো, 1 ক্যারি

পরীক্ষার ফাঁদ: বুলিয়ান OR-এ 1 + 1 = **1**; বাইনারি যোগে 1 + 1 = **10**। একই "+" চিহ্ন, ভিন্ন অর্থ: প্রশ্ন মন দিয়ে পড়ো।

৮ বাইনারি বিয়োগ (ধার পদ্ধতি)

A − B	বিয়োগফল	ধার
0 − 0	0	0
1 − 0	1	0
1 − 1	0	0
0 − 1 = 10 − 1	**1**	**1**

সমাধানকৃত উদাহরণ ✍️: $(11010)_2 - (01111)_2$

বিট 0: 0 − 1 → ধার → 10 − 1 = 1

বিট 1: (1 − 1) − 1 → ধার → 1

বিট 2: (0 − 1) − 1 → ধার → 0

বিট 3: (1 − 1) − 1 → ধার → 1

বিট 4: 1 − 1 − 0 = 0 → $\mathbf{(01011)_2}$ = 11 ✓ (26 − 15)

আরও: $(1010)_2 - (11)_2 = \mathbf{(111)_2}$ (10 − 3 = 7) · $(1101.1)_2 - (110.11)_2 = \mathbf{(110.11)_2}$ (13.5 − 6.75 = 6.75)।

মনে রাখার কৌশল: ধার নিলে কলামে আসে "**একটি ভিত্তি**": বাইনারিতে 2, অক্টালে 8, হেক্সে 16। প্রতিটি উত্তর যাচাই করো: **বিয়োগফল + বিয়োজ্য = বিয়োজন**।

৯ বাইনারি গুণ ও ভাগ

×-এর নিয়ম = AND: 0×0 = 0×1 = 1×0 = 0, 1×1 = 1। ÷ = দীর্ঘ ভাগ; প্রতি ধাপে ভাজক হয় **একবার যায়, নয়তো যায় না**। পদ্ধতি: **শিফট করে যোগ**।

```
      1 1 0 1   (13)
    × 1 0 1     (5)
  ---------
      1 1 0 1   × 1
    0 0 0 0     × 0, শিফট 1
  1 1 0 1       × 1, শিফট 2
  ---------
  1 0 0 0 0 0 1   (65)
```

আংশিক গুণফলগুলো যোগ করো

```
          1 0 0 1   (54 ÷ 6 = 9)
      ------------
110 ) 1 1 0 1 1 0
      - 1 1 0
      -------
        0 1 1 0   01, 011 < 110
                  → ভাগফলে দুটি 0
        - 1 1 0
        -------
            0     ভাগশেষ 0
```

$(110110)_2 \div (110)_2 = \mathbf{(1001)_2}$ · 100111 ÷ 101 = **111, ভাগশেষ 100** (39 ÷ 5 = 7, ভাগশেষ 4)

শিফটের নিয়ম: k ঘর বাঁয়ে শিফট = × 2^k; ডানে শিফট = ÷ 2^k (পূর্ণাংশ)। 1011 << 2 = 101100 (11 → 44)।

পরীক্ষার ফাঁদ: $(1101)_2 \times (101)_2 = \mathbf{1000001}$, 1000011 নয়। দশমিকে মিলিয়ে নাও (13 × 5 = 65 = 64 + 1)।

১০ অক্টাল ও হেক্সাডেসিমাল পাটিগণিত

নিয়ম: কলাম ধরে দশমিকে যোগ করো। যোগফল ≥ ভিত্তি হলে **যোগফল – ভিত্তি লেখো, 1 ক্যারি** (c1)। বিয়োগে ধার নিলে কলামে যোগ হয় **8** (অক্টাল) বা **16** (হেক্স)।

অক্টাল ✍️:

১. 457 + 365: 7 + 5 = 12 = 8 + **4** c1 · 5 + 6 + 1 = 12 → **4** c1 · 4 + 3 + 1 = 8 → **0** c1 → $\mathbf{(1044)_8}$ (303 + 245 = 548)

২. 712 − 345: 2 − 5 → 10 − 5 = **5** · 0 − 4 → 8 − 4 = **4** · 6 − 3 = **3** → $\mathbf{(345)_8}$

৩. $(64)_8 \times 7$: 4×7 = 28 = 3×8 + **4** · 6×7 + 3 = 45 = 5×8 + **5** → $\mathbf{(554)_8}$

হেক্স ✍️:

১. 9A + 7C: A + C = 22 = 16 + **6** c1 · 9 + 7 + 1 = 17 → **1** c1 → $\mathbf{(116)_{16}}$ (154 + 124 = 278)

২. 3A2 − 1F7: 2 − 7 → 18 − 7 = **B** · 9 − F → 25 − 15 = **A** · 2 − 1 = **1** → $\mathbf{(1AB)_{16}}$ (930 − 503 = 427)

৩. $(2A)_{16} \times 3 = 42 \times 3 = 126 = \mathbf{(7E)_{16}}$

মনে রাখার কৌশল: দশমিকে ভাবো, ভিত্তিতে লেখো: "**৭ ছাড়ালে ৮ বাদ, ১৫ ছাড়ালে ১৬ বাদ**", আর 1 ক্যারি।

পরীক্ষার ফাঁদ: $(FF)_{16} + 1 = \mathbf{(100)_{16}} = 256$, আর $(77)_8 + 1 = (100)_8 = 64$। হেক্সে "10" মানে ষোলো।

১১ r-এর ও (r − 1)-এর পরিপূরক

সংজ্ঞা: ভিত্তি r-এ n অঙ্কের সংখ্যা N-এর জন্য: **(r − 1)-এর পরিপূরক = $(r^n - 1) - N$** (প্রতিটি অঙ্ক r − 1 থেকে বিয়োগ); **r-এর পরিপূরক = $r^n - N$** = (r − 1)-এর পরিপূরক + 1। বাইনারিতে **১-এর** ও **২-এর**, দশমিকে **৯-এর** ও **১০-এর** পরিপূরক (complement)।

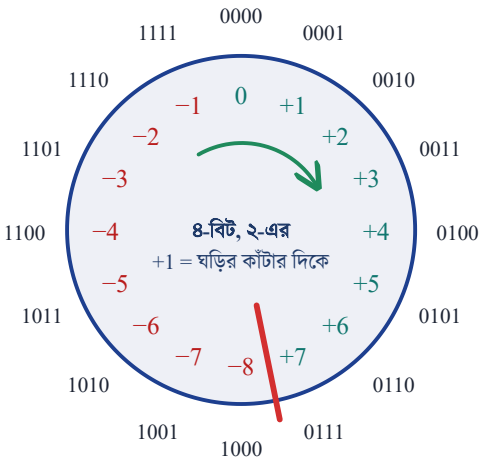
N (8-বিট)	১-এর (উল্টাও)	২-এর (+1)
01011000	10100111	**10101000**
00101101 (45)	11010010	11010011
00000101 (5)	11111010	11111011
00000000	11111111	00000000
দশমিক 0725	9274 (৯-এর)	9275 (১০-এর)

কৌশল: ২-এর পরিপূরকের শর্টকাট: ডান দিক থেকে **প্রথম 1 পর্যন্ত (সেটিসহ) সব বিট হুবহু লেখো**, বাকিগুলো উল্টাও। 0101|1000: "1000" হুবহু, "0101" উল্টে → **1010 1000**।

১২ চিহ্নযুক্ত সংখ্যা ও রেঞ্জ

MSB = চিহ্ন বিট (sign bit; 0 = +, 1 = −)। 8 বিটে **−5** লেখার তিন উপায় (সার্কিটের বিস্তারিত: DLD নোট):

উপস্থাপন	−5 (8-বিট)	n বিটের রেঞ্জ	8-বিট রেঞ্জ	শূন্য
আনসাইনড	(সম্ভব নয়)	$0 \dots 2^n - 1$	0 … 255	1
সাইন-ম্যাগনিচুড	10000101	$-(2^{n-1} - 1) \dots +(2^{n-1} - 1)$	−127 … +127	**2**
১-এর পরিপূরক	11111010	$-(2^{n-1} - 1) \dots +(2^{n-1} - 1)$	−127 … +127	**2**
২-এর পরিপূরক	11111011	$\mathbf{-2^{n-1} \dots +(2^{n-1} - 1)}$	**−128 … +127**	1



লাল দাগ: +7 → −8, এখানেই ওভারফ্লো

MSB-এর ওজন: ২-এর পরিপূরকে MSB-এর ওজন -2^{n-1}। 11111011 = −128 + 64 + 32 + 16 + 8 + 2 + 1 = **−5**।

পরীক্ষার ফাঁদ: 8-বিট ২-এর পরিপূরকে **−128** রাখা যায়, +128 নয়; 10000000-এর ২-এর পরিপূরক 10000000 নিজেই। সাইন-ম্যাগনিচুড ও ১-এর পরিপূরকে একটি বিন্যাস **−0**-তে নষ্ট হয়।

16-বিট: আনসাইনড 0 … 65535, ২-এর পরিপূরক **−32768 … +32767**।

১৩ পরিপূরকের সাহায্যে বিয়োগ

২-এর পরিপূরক পদ্ধতি: A − B = **A + (B-এর ২-এর পরিপূরক)**। **শেষে ক্যারি এলে → বাদ দাও**, ফল ধনাত্মক। **ক্যারি না এলে** → ফল ঋণাত্মক; মান পেতে ফলের ২-এর পরিপূরক নাও।

25 − 13 (8-বিট) ✍️:
25 = 00011001, −13 = 11110011
00011001 + 11110011 = **1** 00001100
ক্যারি বাদ → **00001100 = +12**

13 − 25 (8-বিট) ✍️:
13 = 00001101, −25 = 11100111
00001101 + 11100111 = 11110100 (ক্যারি নেই)
ঋণাত্মক → ২-এর পরিপূরক = 00001100 → **−12**

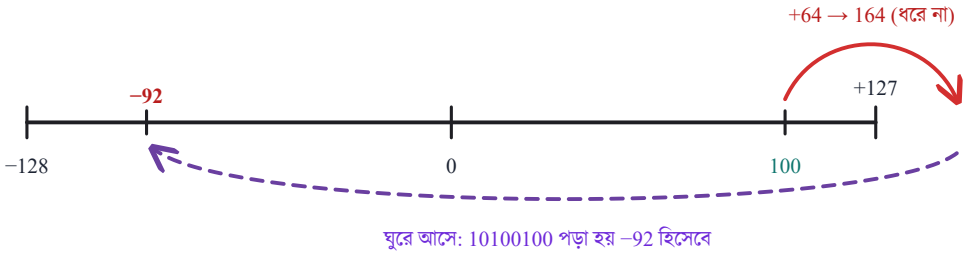
১-এর পরিপূরক পদ্ধতি ✍️: 1011 − 0110 (11 − 6): 1011 + 1001 = **1** 0100 → **এন্ড-অ্যারাউন্ড ক্যারি**: 0100 + 1 = **0101** = 5। ক্যারি না এলে ফল ঋণাত্মক, উত্তর = −(যোগফলের ১-এর পরিপূরক)।

১০-এর পরিপূরক ✍️: 725 − 318 = 725 + 682 = 1407 → ক্যারি বাদ → **407**। (682 = 318-এর ১০-এর পরিপূরক।)

কম্পিউটার কেন ২-এর পরিপূরক ব্যবহার করে: একই অ্যাডারে যোগ ও বিয়োগ · শূন্য **একটিই** · এন্ড-অ্যারাউন্ড ক্যারি লাগে না · চিহ্ন আপনা-আপনি আসে।

১৪ ওভারফ্লো (Overflow)

নিয়ম: আসল ফল n-বিটের রেঞ্জের বাইরে গেলে ওভারফ্লো। ২-এর পরিপূরকে এটি ঘটে শুধু তখন, যখন **একই চিহ্নের দুটি সংখ্যা** যোগ করে **বিপরীত চিহ্নের** ফল আসে; অর্থাৎ **MSB-তে ঢোকা ক্যারি ≠ MSB থেকে বের হওয়া ক্যারি**। বিপরীত চিহ্নের সংখ্যা যোগে ওভারফ্লো **কখনো** হয় না।



উদাহরণ ✍️: 100 + 64: 01100100 + 01000000 = **10100100**: দুটি ধনাত্মক থেকে ঋণাত্মক (−92) → **ওভারফ্লো**।

উদাহরণ ✍️: (−100) + (−64): 10011100 + 11000000 = 1 **01011100** (+92): দুটি ঋণাত্মক থেকে ধনাত্মক → **ওভারফ্লো**।

পরীক্ষার ফাঁদ: **ক্যারি ≠ ওভারফ্লো**। আনসাইনড সংখ্যায় MSB থেকে ক্যারি বের হওয়াই ওভারফ্লো; ২-এর পরিপূরকে ক্যারি বের হওয়া স্বাভাবিক (ওপরের 25 − 13), সেটি শুধু বাদ দেওয়া হয়।

পরীক্ষার আগের রাতের পাতা

তথ্য-তালিকা:

ভিত্তি r: অঙ্ক 0 … r − 1 · $(10)_r = r$

দশমিক ⟶ r: ÷ r উপরে পড়ো · ভগ্নাংশ × r নিচে

$2^8 = 256 \cdot 2^{10} = 1024 \cdot 2^{16} = 65536$

1 + 1 = 10 · 1 + 1 + 1 = 11 · 0 − 1 = 1 (ধার 1)

১-এর = উল্টাও · ২-এর = উল্টাও + 1

২-এর রেঞ্জ: -2^{n-1} … $2^{n-1} - 1$

A − B = A + ২-এর(B); ক্যারি ⟶ বাদ

$N = \Sigma\, d_i \times r^i$ (বিন্দুর পরে $i < 0$)

অক্টাল = ৩ বিট · হেক্স = ৪ বিট (বিন্দু থেকে)

n বিট ⟶ 2^n মান, আনসাইনড সর্বোচ্চ $2^n - 1$

অক্টালে ক্যারি 8-এ · হেক্সে 16-এ

r-এর = $r^n - N$ · (r−1)-এর = $r^n - 1 - N$

S-M / ১-এর: $\pm(2^{n-1} - 1)$, দুটি শূন্য

ওভারফ্লো: $C_{in}(MSB) \neq C_{out}(MSB)$

MCQ-র জন্য সেরা ১২টি এক-লাইনের তথ্য

১. $(1011)_2 = 11$ · $(11001)_2 = 25$

২. $245 = 11110101 = 365_8 = F5_{16}$

৩. $1000 = 3E8_{16} = 1750_8$

৪. $157_8 = 111 = 6F_{16}$

৫. $0.625 = 0.101_2$

৬. 0.1 বাইনারিতে অসীম (non-terminating)

৭. $9A + 7C = 116_{16}$

৮. 1101 × 101 = 1000001

৯. ২-এর(01011000) = 10101000

১০. 8-বিট ২-এর পরিপূরক: −128 … +127

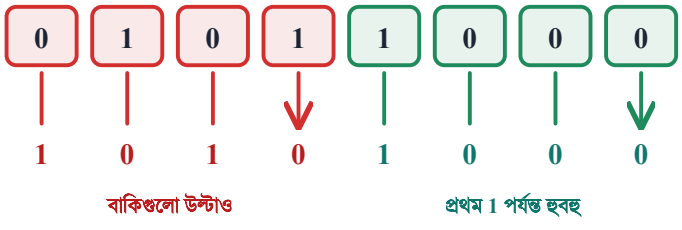
১১. chmod 755 = rwxr-xr-x

১২. 32 বিট = ৮টি হেক্স অঙ্ক

প্রধান ফাঁদ: বুলিয়ান 1 + 1 = 1, বাইনারি 1 + 1 = 10 · ভাগশেষ নিচ থেকে উপরে, ভগ্নাংশের অঙ্ক উপর থেকে নিচে · ভগ্নাংশের গ্রুপে প্যাডিং ডানে · অক্টালে 8 বা 9 নেই · $(FF)_{16} + 1 = 100_{16}$ · 8 বিটে −128 ধরে, +128 ধরে না · ক্যারি ≠ ওভারফ্লো · 1 KB = 1024 B

সব কৌশল একসাথে: ভিত্তি = অঙ্কের সংখ্যা, বড় অঙ্ক = ভিত্তি − 1 · হর্নার: দ্বিগুণ করো, যোগ করো · ভাগ ⟶ উপরে, গুণ ⟶ নিচে · অক্টাল-৩, হেক্স-৪ · ধারে আসে একটি ভিত্তি · ৭ ছাড়ালে ৮ বাদ, ১৫ ছাড়ালে ১৬ বাদ · ২-এর: প্রথম 1 পর্যন্ত হুবহু, বাকি উল্টাও · MSB-এর ওজন -2^{n-1}

লিখিত পরীক্ষার টিপস: ভাগের মই ও গুণের কলাম দেখাও; **প্রতিটি** সংখ্যার নিচে ভিত্তি লেখো; শেষে দশমিকে যাচাইয়ের লাইন দাও ("যাচাই: 3×64 + 6×8 + 5 = 245")।



২-এর পরিপূরকের শর্টকাট, ডান দিক থেকে পড়ো

✓ অনুশীলন MCQ (পরীক্ষার ধাঁচে, ১০ নম্বর)

১. $(1101.01)_2$-এর দশমিক মান:

ক) 13.25 খ) 13.5

গ) 11.25 ঘ) 12.25

২. $(3A)_{16}$-কে অক্টালে লিখলে:

ক) 70 খ) 72

গ) 74 ঘ) 62

৩. 8-বিট ২-এর পরিপূরক সংখ্যার রেঞ্জ:

ক) 0 … 255 খ) −127 … +127

গ) −128 … +127 ঘ) −128 … +128

৪. 10110100-এর ২-এর পরিপূরক:

ক) 01001011 খ) 01001101

গ) 10110100 ঘ) 01001100

৫. $(0.1)_{10}$-এর বাইনারি রূপ:

ক) 0.0001 খ) 0.1

গ) 0.000110011… ঘ) 0.0011

৬. $(457)_8 + (365)_8 = ?$

ক) $(1044)_8$ খ) $(822)_8$

গ) $(1034)_8$ ঘ) $(1144)_8$

৭. কোনটি বৈধ অক্টাল সংখ্যা নয়?

ক) 7770 খ) 1289

গ) 2047 ঘ) 0

৮. 12 বিটে কয়টি ভিন্ন মান রাখা যায়?

ক) 4095 খ) 2048

গ) 1024 ঘ) 4096

৯. 8-বিট ২-এর পরিপূরকে 01100100 + 01000000 (OVF = ওভারফ্লো):

ক) 10100100, OVF নেই

খ) 10100100, OVF

গ) 00100100, OVF

ঘ) 00100100, ক্যারি

১০. $(1000)_{10}$-এর হেক্সাডেসিমাল রূপ:

ক) 3E8 খ) 3F8

গ) 2E8 ঘ) 3E0

উত্তরমালা → ১-ক · ২-খ · ৩-গ · ৪-ঘ · ৫-গ · ৬-ক · ৭-খ · ৮-ঘ · ৯-খ · ১০-ক

লিখিত প্রশ্ন: (ক) $(156.6875)_{10}$-কে বাইনারি, অক্টাল ও হেক্সাডেসিমালে রূপান্তর করো। (খ) 8-বিট ২-এর পরিপূরক ব্যবহার করে 45 − 72 নির্ণয় করো এবং ওভারফ্লো হয় কি না বলো। ৮ নম্বর

১ 156 ÷ 2 → ভাগশেষ 0, 0, 1, 1, 1, 0, 0, 1 → উপরে পড়ে: **10011100**; 0.6875 × 2 → 1.375, 0.75, 1.5, 1.0 → **.1011** ২

২ $(156.6875)_{10}$ = **$(10011100.1011)_2$** → 010 011 100 . 101 100 = **$(234.54)_8$** → 1001 1100 . 1011 = **$(9C.B)_{16}$** ২

৩ 45 = 00101101, 72 = 01001000 → ২-এর(72) = 10111000; 00101101 + 10111000 = **11100101**, শেষে ক্যারি নেই ২

৪ ঋণাত্মক: ২-এর(11100101) = 00011011 = 27 → উত্তর **−27**। চিহ্ন ভিন্ন ছিল, তাই ওভারফ্লো **নেই**। যাচাই: 45 − 72 = −27 ✓ ২

পরীক্ষার আগে যাচাই ✓

- ✓ 2^0 … 2^{16} ও 0 – F-এর বাইনারি মুখস্থ লিখতে পারি
- ✓ পূর্ণাংশ ও ভগ্নাংশ দুদিকেই রূপান্তর করতে পারি
- ✓ ২, ৮ ও ১৬ ভিত্তিতে যোগ-বিয়োগ করতে পারি
- ✓ ২-এর পরিপূরকে বিয়োগ করতে ও ওভারফ্লো চিনতে পারি